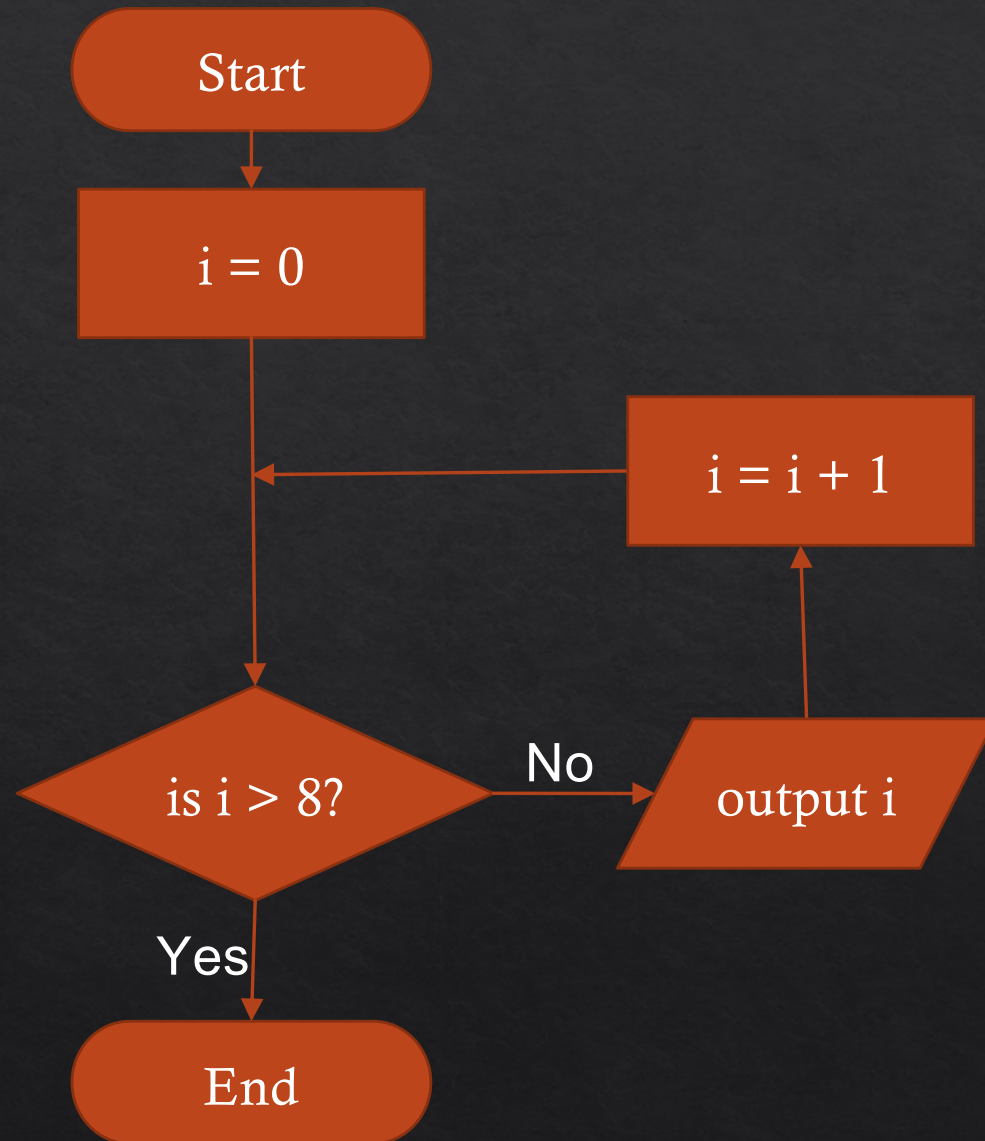


Lecture

Iteration: for-loop





Output:

0
1
2
3
4
5
6
7
8



for-loop is...

- a control structure used to control the flow of execution.
- used to repeat a section of code for a known number of times.

```
for <var_name> in <sequence>:  
    <code in the body of the loop>
```



Why for-loop?

Instead of:

```
print("0")  
print("1")  
print("2")  
print("3")  
print("4")  
print("5")  
print("6")  
print("7")  
print("8")
```

We can code:

```
for i in range(9):  
    print(i)
```



Analysing a for-loop

loop variable name

range(9) refers to sequence of numbers 0 to 8

```
for i in range(9):  
    print(i)
```



Analysing a for-loop

loop variable name

range(9) refers to sequence of numbers 0 to 8

```
for i in range(9):  
    print(i)
```

Output:

0

Set

i to 0

i to 1

i to 2

i to 3

...

i to 7

i to 8

run the code
here



Analysing a for-loop

loop variable name

range(9) refers to sequence of numbers 0 to 8

```
for i in range(9):  
    print(i)
```

Output:

0
1

Set

i to 0

i to 1

i to 2

i to 3

...

i to 7

i to 8

run the code
here



Analysing a for-loop

loop variable name

range(9) refers to sequence of numbers 0 to 8

```
for i in range(9):  
    print(i)
```

Set

i to 0

i to 1

i to 2

i to 3

...

i to 7

i to 8

run the code
here

Output:

0
1
2
3
4
5
6
7
8



Types of for-loops

- a. for-loop with range(n)
- b. for-loop with range(start, stop)
- c. for-loop with range(start, stop, step)
- d. for-loop with string
- e. for-loop using the string index



a. for-loop with range(n)

```
for i in range(n):
```

```
<body>
```

```
for i in range(9):  
    print(i)
```

```
>>> 0  
      1  
      2  
      3  
      4  
      ...  
      7  
      8
```



b. for-loop with range(start, stop)

```
for i in range(start, stop):  
    <body>
```

```
for i in range(3, 9):  
    print(i)
```

```
>>> 3  
      4  
      5  
      6  
      7  
      8
```



c. for-loop with range(start, stop, step)

```
for i in range(start, stop, step):  
    <body>
```

```
for i in range(3, 9, 2):  
    print(i)
```

```
>>> 3  
      5  
      7
```



d. for-loop with string

```
for char in 'Singapore':  
    print(char)
```

```
>>> S  
      i  
      n  
      g  
      a  
      p  
      o  
      r  
      e
```

Use this method if
you do not need
the index of the
character



e. for-loop using string index

```
string = 'Singapore'
for i in range(len(string)):
    print(i, string[i])
```

```
>>> 0 S ← string[0]
      1 i ← string[1]
      2 n ← string[2]
      3 g ← string[3]
      4 a ← string[4]
      5 p ← string[5]
      6 o ← string[6]
      7 r ← string[7]
      8 e ← string[8]
```

Since $\text{len}(\text{string}) = 9$

$\text{range}(\text{len}(\text{string}))$

$\Rightarrow \text{range}(9)$



e. for-loop using string index

```
string = 'Singapore'  
for i in range(0, len(string), 2):  
    print(string[i])
```

```
>>> S ← string[0]  
    n ← string[2]  
    a ← string[4]  
    o ← string[6]  
    e ← string[8]
```



len(string) = 9



Can we break or skip in a for-loop?



What does break do?

Code:

```
for i in range(9):  
    print(i)  
    if i == 3:  
        break  
print("Exit loop")
```

Output:

```
0  
1  
2  
3  
Exit loop
```

break terminates the execution of the iteration, and exits the for-loop



What does continue do?

Code:

```
def print_odd():  
    for i in range(9):  
        if i % 2 == 0:  
            continue  
        print(i)  
    print("Exit loop")  
print_odd()
```

Output:

```
1  
3  
5  
7  
Exit loop
```

`continue` skips the current iteration
and continues the execution with the next iteration.



What is factorial?

$$0! = 1 \text{ (By definition)}$$

$$1! = 1$$

$$2! = 1 * 2$$

$$3! = 1 * 2 * 3$$

$$4! = 1 * 2 * 3 * 4$$

...

$$20! = 1 * 2 * 3 * \dots * 19 * 20$$

$$n! = 1 * 2 * 3 * \dots * n$$



Write the factorial function

◆ Start with 1, multiply by 2, multiply by 3, ... until n

◆ Think of a plan:

Store the current result somewhere?

Use a loop?

```
def factorial(n):  
    result = 1  
    #Loop from 1 to n. Remember to include n.  
    # inside the body of the loop, we need to  
    # update the result by multiplying with n  
    return result
```



Write the factorial function

```
def factorial(n):  
    result = 1  
    for i in range(1, n+1):  
        result = result * i  
    return result
```

```
>>> factorial(6)
```

```
720
```

i	result
	1
1	1
2	2
3	6
4	24
5	120
6	720

return 720



Common Mistakes: for-loop

Qn: Use iteration to sum the even numbers from 0 to n, inclusive of n.

```
def even_sum(n):
```

```
    sum = 0
```

```
    for i in range(10):
```

```
        if i % 2 == 0:
```

```
            sum = sum + i
```

```
    return sum
```

```
>>> even_sum(10)
```

1. Missing colon

n+1

2. Input argument n was not even used in the code. range was hardcoded to 10

3. Values of i are off by 1

4. Do we return inside the loop body? Or outside?



Debugging a for-loop

Qn: Write a function `invest(amount, years)` that uses iteration to calculate the `amount` of money obtained after investing it with a 5% compound interest rate over a number of `years`.

Understand it with a simple example:

This is a table to understand how investing \$100 can accumulate over 5 years, with a yearly compounding interest rate of 5%.

1	105.0
2	110.25
3	115.76
4	121.55
5	127.63



Debugging a for-loop

Can you locate the error in the program code?

```
INTEREST_RATE = 0.05          #compounded yearly interest
```

```
def invest(amount, year):  
    for i in range(1, year):  
        amount = amount * INTEREST_RATE  
    return round(amount, 2)
```

```
>>> invest(100, 5)
```

```
In [3]: 1 invest(100,5)  
Out[3]: 0.0
```

Why is my \$100
investment giving me \$0?



Debugging a for-loop

If you include a print statement,
it may help in debugging

```
INTEREST_RATE = 0.05          #compounded yearly interest
```

```
def invest(amount, year):  
    for i in range(1, year):  
        amount = amount * INTEREST_RATE  
        print(i, amount)  
    return round(amount, 2)
```

1	5.0
2	0.25
3	0.01
4	0.0

```
>>> invest(100, 5)
```

Oh, I forgot to add the principal amount,
and only processed 4 years, not 5



Debugging a for-loop

Once we understand the problem in the program code, we can easily correct it.

```
INTEREST_RATE = 0.05          #compounded yearly interest
```

```
def invest(amount, year):  
    for i in range(1, year + 1):  
        amount = amount + amount * INTEREST_RATE  
        print(i, amount)  
    return round(amount, 2)
```

1	105.0
2	110.25
3	115.76
4	121.55
5	127.63

```
>>> invest(100, 5)
```

